

Sunny Tea House AI 智能评价生成系统 · 技术与设计文档

👤 薄皓元 (Haoyuan Bo) ⌚ 实际耗时: 约 2.5 小时 (借助 AI 编程工具完成) ⚡ 体验地址: sunny-tea.lumiere-private.com

📄 交付标准速览: 1. AI工具提效 | 2. Prompt调优与复盘预案 | 3. 企微Webhook流水线 | 4. Key安全与成本 | 5. 交互闭环 | 6. 耗时分析

1. 💡 AI 辅助编程工具使用策略与提效心得

本次任务官方预期耗时约为 **4~6 小时** (核心任务 4h + 附加题 1~2h)。在实际全栈开发中,我全程使用 **Cursor / Claude Code** 辅助结对编程,耗时 **约 2.5 小时** 完成了从需求解构、双端 Prompt 调优、移动端 H5、Next.js Serverless 代理、企微 Webhook 异步流水线、自由打字 Agent 升级到独立域名上线的全部闭环:

- 需求解构与 Schema-First (架构先行)**: 优先让 AI 梳理出严格的 TypeScript 接口定义与 Webhook JSON Payload 规范,前后端传参一次性对齐,避免了返工修改。
- 移动端 CSS 样式与 Apple HIG 规范调试**: 以前手写移动端 H5 自适应、点击热区 ($\geq 44\text{px}$) 和高对比度浅灰/纯白卡片留白可能要花 1~2 小时。这次让 AI 输出 Tailwind 样式类名,快速把自适应高度和响应式布局调顺了。
- 接口容错与流式调试**: 在开发大模型接口与 Webhook 时,直接把报错日志丢给 AI 进行根因分析,秒级定位数据分块边界与 JSON 异常。

2. 🌐 跨平台、跨语种 Prompt 设计思路与调优过程

2.1 踩坑记录: 跨语言标签污染问题 (真实工程沉淀)

遇到的问题 (V1.0): 一开始直接把前端用户选中的中文标签 (如 ['服务好', '出餐快']) 拼进英文 Prompt 里丢给大模型,导致模型在生成英文评价时强行保留中文标签,输出了 "The 服务好, 出餐快 really stood out..."。

原因分析: 大模型在缺少明确的多语言引导时,会把上下文里的非英文词当成专有名词原样保留。

解决办法 (V3.0 定稿):

- 代码层语义映射字典 (TAG_EN_MAP)**: 在发送给大模型前,先在后端字典将中文标签映射为地道英文短语 (例如 '服务好' -> 'friendly and welcoming staff', '出餐快' -> 'super fast turnaround');
- System Prompt 绝对负约束**: 加入强力指令 "You must output ONLY in natural, fluent American English. NEVER output any Chinese characters.";
- 后端正则保底守卫**: 在响应返回前增加正则检查,若意外检测到中文字符则自动激活纯净英文备用逻辑,彻底根除多语言混杂。

2.2 跨平台 Prompt 差异化矩阵

维度	🌐 Google Review (北美本地口碑)	📌 小红书 (RED 爆款种草)
目标受众	北美本地居民、硅谷工程师、Google Maps 真实食客	湾区华人、留学生、探店打卡群体
语言与口吻	100% 地道美式英语 (American English)，客观、冷静、普通消费者视角	纯中文简体，热情闺蜜安利感、真诚打卡、网络化情绪价值
严格禁止	严禁生硬机器翻译腔（如 <code>"I had the pleasure..."</code> ）和任何中文字符	严禁生硬官话、严禁大段文字堆叠，必须空行留出呼吸感
标志元素	口碑词： <code>"Super chewy boba"</code> , <code>"Fast turnaround"</code> , <code>"Well balanced sweetness"</code>	灵动 Emoji (🍹🌟🍰💖🔥)、空行呼吸感排版、3个精准 Tag
长度控制	严格控制在 50~70 Words (单词) ，以 Word Count 计量	120-150 字，以字符计数，分为 3-4 个精简微段落

💡 **长度控制底层机制（加分亮点）：**为了解决 Google 英文模式下字数忽多忽少的问题，我在 Prompt 中并未采用保守的 `max_tokens` 粗暴截断（这会破坏句子的语法完整性导致断尾），而是启用了 `stop_sequences`（结束序列）配合 Few-shot 样本的长度锚定，确保输出在 50~70 个单词时自然完整收尾。

2.3 线上复盘与现场调 Prompt 的预案

代码里把 Prompt 逻辑全部拆到了 `src/app/api/generate/route.ts` 头部，复盘现场如果需要微调，改起来很快：

- **场景 A（改成温和差评/改进建议）：**只需在 System Prompt 里把 Tone 改为 *"Constructive & polite customer feedback, mentioning long wait time"*，即可秒级切换；
- **场景 B（增加指定单品或季节限定）：**代码里入参已经完全解耦，直接把饮品名传进 `userPrompt` 拼接即可；
- **Agent 自由打字与选词融合：**支持在步骤 1 直接输入任意打卡描述（不选标签也能生成），Prompt 会自动提炼其中的闪光点并转化为地道评价。

3. 🤖 附加题：企业微信群机器人自动化 workflow（Webhook）

当顾客生成好评文案后，系统在后台静默触发异步非阻塞后台任务：

1. 大模型秒级提炼出 **20 字以内的中文摘要** 与 **情感倾向判定**；
2. 自动生成一段具有品牌温度的 **《50字给奶茶店老板的亲切感谢回复草稿》**；
3. 组装为结构化企业微信 Markdown 卡片，实时推送到门店运营群。

- 💡 **兼顾演示与真实推送的设计：**
- **异步非阻塞：**Webhook 推送用 `try...catch` 包裹在后台异步执行，即便推送失败或网络超时，也不会卡住前端顾客获取评价的主请求（主流程依然秒级返回 200）；

- **环境切换**：如果在环境变量里配置了真实的 `WEWORK_WEBHOOK`，它会真实发到企微信群；如果不配，系统就会在后台记录拼装好的数据，前端底部也做了一个极简折叠抽屉，点开就能直接看到已组装的 AI 摘要与回复草稿。

```
{
  "msgtype": "markdown",
  "markdown": {
    "content": "### 🍵 Sunny Tea House 新评价通知\n> **来源平台**：小红书\n> **用户标签**：服务好 / 出餐快\n**评价内容**"
  }
}
```

4. 🛡️ API Key 服务端安全防护与用量成本优化（加分考察点）

🔒 纯前端方案的安全隐患与服务端隔离架构

题目特别指出：“纯前端直连方案会使 API Key 暴露在浏览器中”。在工业级开发中，前端直连暴露 Key 属于重大安全漏洞。为此，本项目坚决采用 **Next.js Serverless Edge Proxy 服务端代理架构**：客户端只负责把用户选的标签和平台通过 `POST /api/generate` 传给服务端；服务端在环境变量 `process.env.GROQ_API_KEY` 里读取 Key 调用大模型，浏览器端抓包只能看到业务数据流，看不到任何 Key。

⚡ 用量成本与极速响应设计

基于 Groq LPU 架构，模型推理速度达到 **500+ Tokens/秒**，首字时延 (TTFT) 压缩至 **200~300ms**，点击后秒级出字；免费层提供每天 **14,400 次高并发请求**，单次请求成本接近 **\$0.0000**，对于 Demo 和后续测试完全够用，不需要担心成本超支。

5. 📱 移动端 H5 交互设计、唤起容错与产品体验闭环

- **步骤 1 Agent 自由输入 + 步骤 2 标签多选**：步骤 1 支持自由打字输入用餐感受（不选标签也能直接生成）；步骤 2 提供 5 个高频消费感受标签并支持自定义添加 Tag，**选满 2 个标签后其余未选标签自动置灰禁用**（`opacity-40 cursor-not-allowed pointer-events-none`），物理杜绝超选。
- **“生成中...” 与 “换一批” 加载态**：点击生成后按钮立即禁用并显示旋转 Loading 图标；点击“🔄 换一批”带旋转动效，毫秒级刷新另一条不同视角的文案。
- **文本框高度自适应与中英文分立计量**：通过 `useEffect` 与 `scrollHeight` 动态调整 Textarea 高度，内容完整舒展平铺；**Google 英文显示 Word Count**（如 58 Words），**小红书显示字符数**（如 142 字）。
- **一键复制与绿色 Toast 瞬时反馈**：复制成功后在屏幕中央弹出高对比度绿色 Toast（`✅ 已复制到剪贴板，即将跳转...`），并在 1.5 秒后平滑淡出。
- **智能 Deep Linking 容错跳转**：对 Google 直跳真实 Google Maps 网页评价入口；对小红书无缝跳转官方移动端 Scheme / 网页搜索打卡入口。

6. 🕒 项目实际耗时与效率对比分析（2.5 小时 vs 5~6 小时）

开发环节	传统纯手工耗时 预估	本次实际耗时 (借助 AI 工具)	做的事情
需求解构与架构设计	约 50 分钟	约 20 分钟	梳理 5 个关键点，确立 Next.js + Serverless 架构
Prompt 设计与跨语种调优	约 80 分钟	约 35 分钟	调优去 AI 味，解决中文标签污染，建立 <code>TAG_EN_MAP</code> 字典
移动端 H5 界面与微交互	约 100 分钟	约 35 分钟	用 Tailwind 调好移动端适配、自适应高度和按钮热区
Serverless API 与 Webhook 流水线	约 60 分钟	约 25 分钟	写好后端代理、错误兜底和企微 Markdown 消息组装
V2 自由打字 Agent 与换一批升级	约 50 分钟	约 20 分钟	支持纯打字生成、自定义 Tag 与 0.9 高随机度重刷
全链路联调、加固与独立域名部署	约 40 分钟	约 15 分钟	Vercel 关联 GitHub，绑定独立域名上线验证
总计项目耗时	约 5.5 小时	约 2.5 小时	双版本深度迭代，全流程高效交付